COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	16/07/2026	**DATE SUBMITTED:**	16/07/2026
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Sir. Jimlord Quejado

ACTIVITY NO. 2 **Arrays_Pointers_and Dynamic Memory Alloc**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
            int studentAge;

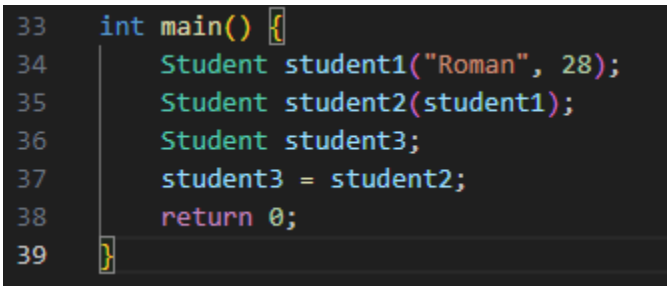
    public:
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called. " << std::endl;
        };

    ~Student () {
        std::cout << "Destructor Called." << std::endl;
    }

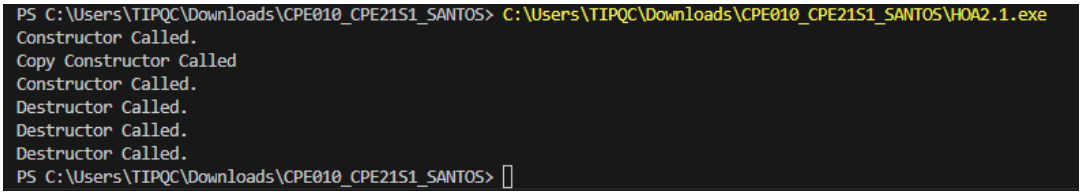
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;


    }

    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

	
Observation	The program follows the big three Constructor, Destructor, and Copy Constructor. As the code shows after every object is constructed the program prints that the object is constructed. Then after all the objects are constructed they are then all destroyed which is then called by the program.

```
33  int main() {
34      Student student1("Roman", 28);
35      Student student2(student1);
36      Student student3;
37      student3 = student2;
38      return 0;
39  }
```

Output	

```
PS C:\Users\TIPQC\Downloads\CPE010_CPE21S1_SANTOS> C:\Users\TIPQC\Downloads\CPE010_CPE21S1_SANTOS\HOA2.1.exe
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Downloads\CPE010_CPE21S1_SANTOS>
```

Table 2-1. Initial Driver Program

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
            int studentAge;

    public:
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called. " << std::endl;
        };

    ~Student () {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;

    }

    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

```cpp
int main() {
const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

return 0;
}
```

Observation

This program used the same class as the previous code but the int main includes an array. With the values inside the array are constructed into objects one by one which is then called by the program. Then after every object is created it is then destroyed after the program is done which is then called in the output.

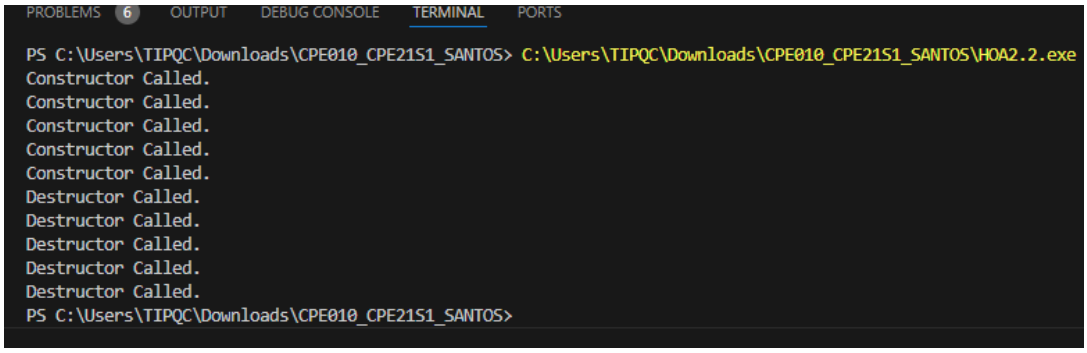
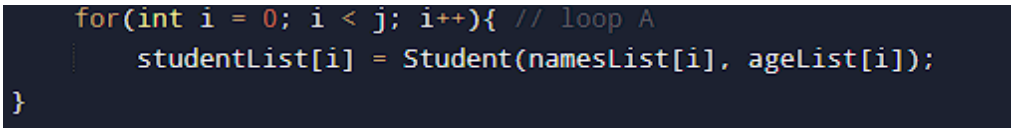
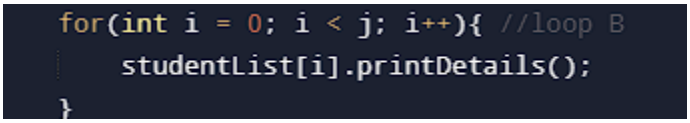
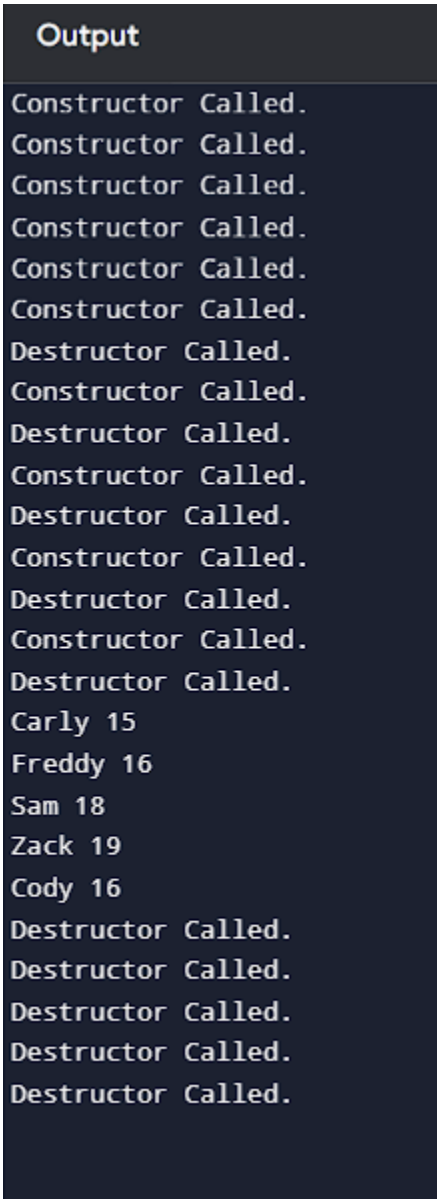
Output	

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	In this loop the names and ages of the students were individually called and assigned to studentList. The values of the students are now stored within studentsLists which is then going to be printed out by loop B.
Loop B	
Observation	This loop was created in order to print out the values that were called in studentList.The loop iterated 5 times before stopping as the values called inside studentList is just 5.

```
    for(int i = 0; i < j; i++){ // loop A
        studentList[i] = Student(namesList[i], ageList[i]);
}
```

```
    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
    }
```

Output	
Observation	The names and numbers of the students were printed out by the program. Because of the loops created the information inside the lists were printed out into the output accompanied by the construction and destruction of elements.

Modifications	
Observation	Loop A was modified in order to prevent any memory leakage as the code provided only used new and did not include a delete so this will cause a memory leak, because the objects in the heap are orphaned. So doing this modification will ensure that there will be no memory leak while also being an efficient way to assign objects into studentList.

Table 2-3. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
#include <iostream>
#include <string>
#include <iomanip>

// Base Class: Item
class Item {
protected:
    std::string name;
    double price;
    int quantity;

public:
    // Default / Parameterized Constructor
    Item(std::string n = "", double p = 0.0, int q = 0)
        : name(n), price(p), quantity(q) {}

    // Virtual Destructor
    virtual ~Item() {}

    // Copy Constructor
    Item(const Item& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    Item& operator=(const Item& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
```

```
        return *this;
    }

    // Getters
    std::string getName() const { return name; }
    double getPrice() const { return price; }
    int getQuantity() const { return quantity; }

    // Member function to calculate sum for the item
    double calculateSum() const {
        return price * quantity;
    }

    // Virtual function to display item details
    virtual void display() const {
        std::cout << std::left << std::setw(12) << name
                  << " | Price: PHP " << std::setw(5) << price
                  << " | Qty: " << std::setw(4) << quantity
                  << " | Total: PHP " << calculateSum() << "\n";
    }
};

// Problem 1: Fruit Class
class Fruit : public Item {
public:
    // Constructor
    Fruit(std::string n = "", double p = 0.0, int q = 0)
        : Item(n, p, q) {}

    // Destructor
    ~Fruit() override {
        std::cout << "[Destructor] Fruit '" << name << "' destroyed.\n";
    }
```

```cpp
    // Copy Constructor
    Fruit(const Fruit& other) : Item(other) {}

    // Copy Assignment Operator
    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            Item::operator=(other);
        }
        return *this;
    }

    void display() const override {
        std::cout << "[Fruit]     ";
        Item::display();
    }
};

// Problem 1: Vegetable Class
class Vegetable : public Item {
public:
    // Constructor
    Vegetable(std::string n = "", double p = 0.0, int q = 0)
        : Item(n, p, q) {}

    // Destructor
    ~Vegetable() override {
        std::cout << "[Destructor] Vegetable '" << name << "' destroyed.\n";
    }

    // Copy Constructor
    Vegetable(const Vegetable& other) : Item(other) {}
```

```
101     Vegetable& operator=(const Vegetable& other) {
102         if (this != &other) {
103             Item::operator=(other);
104         }
105         return *this;
106     }
107
108     void display() const override {
109         std::cout << "[Vegetable] ";
110         Item::display();
111     }
112 };
113
114 // Problem 3: Function to calculate total sum of all non-null items in the list
115 double TotalSum(Item* groceryList[], int size) {
116     double total = 0.0;
117     for (int i = 0; i < size; ++i) {
118         if (groceryList[i] != nullptr) {
119             total += groceryList[i]->calculateSum();
120         }
121     }
122     return total;
123 }
124
125 int main() {
126     const int LIST_SIZE = 4;
127
128     // Problem 2: Create dynamic array GroceryList storing polymorphic pointers
129     Item* GroceryList[LIST_SIZE];
```

```
131     // Dynamically allocate objects based on the table
132     GroceryList[0] = new Fruit("Apple", 10.0, 7);
133     GroceryList[1] = new Fruit("Banana", 10.0, 8);
134     GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
135     GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);
136
137     // Problem 2: Display details of all items
138     std::cout << "--- JENNA'S GROCERY LIST ---\n";
139     for (int i = 0; i < LIST_SIZE; ++i) {
140         if (GroceryList[i] != nullptr) {
141             GroceryList[i]->display();
142         }
143     }
144
145     // Problem 3: Calculate and display the grand total sum
146     std::cout << "\n----------------------------------------\n";
147     double grandTotal = TotalSum(GroceryList, LIST_SIZE);
148     std::cout << "Grand Total: PHP " << grandTotal << "\n";
149     std::cout << "----------------------------------------\n\n";
150
151     // Problem 4: Delete Lettuce (at index 3) and de-allocate memory
152     std::cout << "--- DELETING LETTUCE ---\n";
153     for (int i = 0; i < LIST_SIZE; ++i) {
154         if (GroceryList[i] != nullptr && GroceryList[i]->getName() == "Lettuce") {
155             delete GroceryList[i];      // De-allocate dynamically allocated Lettuce
156             GroceryList[i] = nullptr;   // Prevent dangling pointer
157             break;
158         }
159     }
160
```

```
161     // Display list again after deletion & show updated total
162     std::cout << "\n--- GROCERY LIST AFTER DELETION ---\n";
163     for (int i = 0; i < LIST_SIZE; ++i) {
164         if (GroceryList[i] != nullptr) {
165             GroceryList[i]->display();
166         } else {
167             std::cout << "[Empty Slot]\n";
168         }
169     }
170
171     std::cout << "\nUpdated Grand Total: PHP " << TotalSum(GroceryList, LIST_SIZE) << "\n\n";
172
173     // Clean up remaining allocated memory
174     std::cout << "--- CLEANING UP REMAINING MEMORY ---\n";
175     for (int i = 0; i < LIST_SIZE; ++i) {
176         if (GroceryList[i] != nullptr) {
177             delete GroceryList[i];
178             GroceryList[i] = nullptr;
179         }
180     }
181
182     return 0;
183 }
```

Output

```
Output
--- JENNA'S GROCERY LIST ---
[Fruit]     Apple        | Price: PHP 10    | Qty: 7    | Total: PHP 70
[Fruit]     Banana       | Price: PHP 10    | Qty: 8    | Total: PHP 80
[Vegetable] Broccoli     | Price: PHP 60    | Qty: 12   | Total: PHP 720
[Vegetable] Lettuce      | Price: PHP 50    | Qty: 10   | Total: PHP 500

----------------------------------------
Grand Total: PHP 1370
----------------------------------------

--- DELETING LETTUCE ---
[Destructor] Vegetable 'Lettuce' destroyed.

--- GROCERY LIST AFTER DELETION ---
[Fruit]     Apple        | Price: PHP 10    | Qty: 7    | Total: PHP 70
[Fruit]     Banana       | Price: PHP 10    | Qty: 8    | Total: PHP 80
[Vegetable] Broccoli     | Price: PHP 60    | Qty: 12   | Total: PHP 720
[Empty Slot]

Updated Grand Total: PHP 870

--- CLEANING UP REMAINING MEMORY ---
[Destructor] Fruit 'Apple' destroyed.
[Destructor] Fruit 'Banana' destroyed.
[Destructor] Vegetable 'Broccoli' destroyed.
```

Define a base Item class and two derived classes named Fruit and Vegetable, which each implement the required Rule of Three constructors, assignment operators, and destructors. An array of polymorphic Item pointers (GroceryList) is created in main, allowing functions like display and calculateSum to properly distinguish fruits from vegetables while computing item totals. A dedicated TotalSum function iterates over the pointer array to accumulate the overall cost of all items, after which the program explicitly locates and deletes the "Lettuce" instance using delete and sets it to nullptr.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.